Nearest Neighbor Prototyping using K-Means Medoids (CSE 251A)

Ryan Livengood [1]

1. Abstract

This paper discusses a strategy to find an informative prototype of the MNIST dataset using Lloyd's K-Means clustering. First, K-means++ is performed per-class to spread cluster centers across class regions. Then Lloyd's K-Means is run to find M clusters. The centroids of these clusters are mapped to Medoids and used as prototype points. These prototype points effectively cover the most informative clusters of data, achieving an increase in accuracy over uniform random selection.

2. Pseudocode

The algorithm takes a dataset of data points X and labels y, in addition to a size M of the desired prototype. The data is split by label to avoid class imbalance, instead prioritizing even spread over each class. Since there are 10 classes in the MNIST dataset, we create variable k = M//10, then find k medoids for every class. The medoids are found using a combination of K-means++, Lloyd's clustering algorithm, and argmin. First, the centroids are found using Lloyd's algorithm, which returns the center of every cluster. The medoids are the points closest to these centroids (since centroids are not necessarily real points), and is the point x_i closest to the centroid. These medoids are all added to a list and returned as the prototype.

Algorithm 1 K-means Prototype

Input: data x, label y size m
Initialize medoids =[].
for $i = 0$ **to** 9 **do**
  $x_i, y_i = x, y$ where $y == i$
  centroids = $Lloyds(x_i, y_i)$
  medoids += $argmin_{x_i}$(dist(centroids, x_i))
end for
Return medoids

Lloyd's algorithm is a helper in this algorithm. It initializes it's centers using K-means++. It then assigns each data point to it's closest center, and moves every center to the mean of it's assigned points. This is repeated until none of the centers move. It is performed as follows:

Algorithm 2 Lloyd's K-means Clustering

Input: data $x_i \dots x_n$, number of clusters k
Initialize: centers $C = \{c_1, \dots, c_k\}$ using K-means++
repeat
  for $i = 1$ **to** n **do**
    Assign x_i to cluster
    $a_i \leftarrow \arg\min_{j \in \{1,\dots,k\}} \|x_i - c_j\|^2$
  end for
  for $j = 1$ **to** k **do**
    Update center
    $c_j \leftarrow \frac{1}{|\{i:a_i=j\}|} \sum_{i:a_i=j} x_i$
  end for
until centers do not change

K-means++ is our initialization algorithm. Without initializing using K-means++, centers may be chosen very close to one another, which will inaccurately assign clusters close to the centers instead of where structure truly exists. It works by randomly selecting the first center, then weighing each point according to how far it is from every other center, and picking one of the points at random. This is to avoid always choosing the furthest point, which could leave our centers on an outlier point, far away from any true cluster.

Algorithm 3 K-means++ Initialization

Input: data $x_1, \dots, x_n$, number of clusters k
Choose first center c_1 at random from $\{x_i\}$
for $j = 2$ **to** k **do**
  for $i = 1$ **to** n **do**
    $D(x_i) \leftarrow \min_{\ell<j} \|x_i - c_\ell\|^2$
  end for
  Choose c_j from $\{x_i\}$ with probability

$$P(x_i) = \frac{D(x_i)}{\sum_{r=1}^{n} D(x_r)}$$

end for
Output: initial centers $C = \{c_1, \dots, c_k\}$

3. Experimental Results

After testing values of M = 100, 1000, 2500, 5000, 7500, 10000, 20 times each on both the k-means prototype method and the random method, a clear improvement is seen by using k-means. Below is a table of the mean classification error rate of both methods, calculated using 1 - accuracy, where accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$. Both algorithms were ran 20 times at each M because confidence intervals tended to stabilize around that number of trials.

Table 1. Classification error rate for random prototype and K-means prototype on various prototype sizes.

PROTOTYPE SIZE	RANDOM	K-MEANS	BETTER?
100	29.3± 0.8	13.4± 0.1	√
1000	11.4± 0.2	7.2± 0.1	√
2500	8.3± 0.1	5.8± 0.1	√
5000	6.4± 0.1	5.1± 0.1	√
7500	5.6± 0.1	4.7± 0.1	√
10000	5.2± 0.1	4.5± 0.04	√

The mean classification error rate was calculated by doing $\mu_x = \frac{1}{||x||}\sum_i x_i$ for every classification error rate grouped by the prototype size. The Confidence interval was calculated by first finding the standard deviation $\sigma = \sqrt{\frac{1}{||x||-1}\sum_i^n (x_i - \mu_x)^2}$. Standard error was calculated via $SEM = \frac{\sqrt{\sigma}}{||x||}$. Confidence Interval was then calculated using the formula $1.96 \cdot SEM$, because 1.96 is the number of standard deviations required to capture 95% of the area under the standard normal distribution curve.

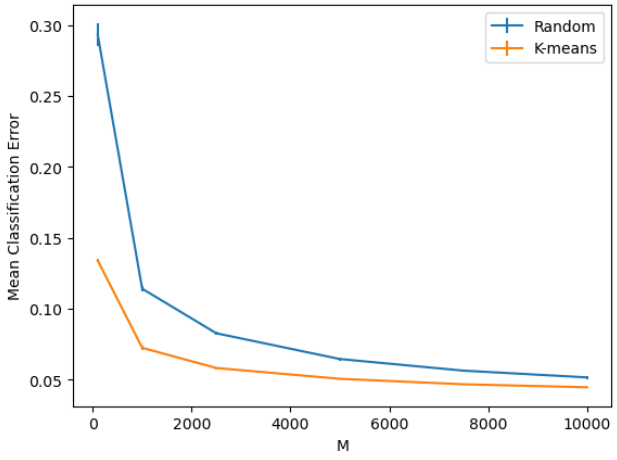

Figure 1. Mean Classification Error percentage vs size of prototype M, Random and K-means

Critical Evaluaion

This method is a clear improvement over the random selection, with an average 28% decrease in classification error rate. One clear drawback of the K-Means selection method is the runtime. Doing 20 runs of M=10,000 using random selection takes 3.5 seconds, while the K-Means selection takes 12 minutes and 41 seconds. If several queries are then run on this prototype, the prototype construction cost will be amortized. Given the decrease in classification error rate, it is likely worth it to run this algorithm, especially on much smaller prototype sizes (M=100 had 54% decrease in classification error rate).

To improve the runtime, a few methods could be introduced. First, Principal Component Analysis could be run to shrink the number of dimensions to 15-30, speeding up the K-means algorithm while generally keeping distance structure. Second, K-means could be run on a smaller batch of the data without much loss of accuracy because MNIST data tends to be tightly clustered.
To improve accuracy, I would like to look into Condensed Nearest Neighbor, where points in the majority class instance are removed to balance out the data, revealing class boundaries. This hybrid approach will help for places with tricky decision boundaries like 4 and 9, where k-means assumes that all decision clusters are equally sized and shaped.

Acknowledgements

LLMs were used to help with the creation of pseudocode for Lloyd's and K-means++ in LaTeXformat. They were also used to help determine the correct calculation of confidence interval.

References

Singh, Ashutosh , *Prototype selection for Nearest Neighbors* Medium, 2019

Abhishek *CNN (Condensed Nearest Neighbors)*, Medium, 2021